



T.C. SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
SİSTEM PROGRAMLAMA DERSİ PROJE RAPORU

Öğrenci Numarası: B231210033-B231210095

Öğrenci Adı Soyadı: Ali Samet Armağan-Zeynep Pakize Uzman

Ders Grubu: 1.öğretim B grubu

Ödev Konusu: Linux Ortamında C Dili ile Sıkıştırmasız Arşivleme Aracı
Geliştirilmesi

GitHub Linki: <https://github.com/BilgeFeo/ArchiveFile-tarsau>

Bu rapor, Bilgisayar Mühendisliği Sistem Programlama dersi kapsamında geliştirilen "tarsau" adlı arşivleme aracının tasarım kararlarını, uygulama detaylarını ve test sonuçlarını açıklamaktadır.

tarsau; tar, rar ve zip gibi popüler arşivleme araçlarına benzer şekilde çalışan, ancak sıkıştırma yapmayan sade bir arşivleme programıdır. Program yalnızca ASCII metin dosyalarını kabul ederek bunları tek bir .sau uzantılı arşiv dosyasında birleştirir ve gerektiğinde geri açabilir.

Projenin Amacı

Projenin temel amacı, Linux/Unix ortamında C programlama dili kullanarak dosya sistemi işlemlerini, komut satırı argüman ayrıştırma ve ikili/metin dosya formatlarını öğrenmek ve uygulamaktır. Bu süreçte aşağıdaki konular ele alınmıştır:

- POSIX sistem çağrıları: stat(), fopen(), fread(), fwrite(), chmod(), mkdir()
- Komut satırı argüman ayrıştırma ve doğrulama
- Özel ikili dosya formatı tasarımı
- Modüler C kodu organizasyonu ve Makefile kullanımı
- Git ile sürüm kontrolü

Proje Yapısı ve Dosya Organizasyonu

Proje, okunabilirliği ve bakım kolaylığını artırmak amacıyla modüler bir klasör yapısında düzenlenmiştir:

Dosya / Dizin	Açıklama
src/main.c	Programın giriş noktası; argüman parse ve mod yönlendirme
src/parser.c	Komut satırı argümanlarını ayrıştıran modül
src/pack.c	Arşiv oluşturma (-b modu) işlevleri
src/unpack.c	Arşiv açma (-a modu) işlevleri
src/utls.c	Yardımcı fonksiyonlar (ASCII kontrolü, dizin oluşturma vb.)
include/tarsau.h	Ortak veri yapıları ve sabitler
include/pack.h	pack modülü başlık dosyası
include/unpack.h	unpack modülü başlık dosyası
include/parser.h	parser modülü başlık dosyası
include/utls.h	utls modülü başlık dosyası
Makefile	Derleme kuralları
bin/tarsau	Derlenmiş çalıştırılabilir dosya
lib/*.o	Ara nesne dosyaları

Veri Yapıları ve Sabitler

tarsau.h başlık dosyasında tanımlanan temel veri yapıları şunlardır:

FileEntry Yapısı

Her arşiv dosyasına ait meta verileri tutar:

```
typedef struct {
    char    name[256];          /* dosya adı (basename) */
    char    fullpath[512];      /* tam yol (okuma için) */
    mode_t  permissions;       /* orijinal izin bitleri */
    size_t  size;               /* dosya boyutu (byte) */
} FileEntry;
```

Archive Yapısı

Tüm dosya kayıtlarını ve dosya sayısını bir arada tutar:

```
typedef struct {
    FileEntry files[MAX_FILES]; /* en fazla 32 dosya */
    int        file_count;      /* gerçek dosya sayısı */
} Archive;
```

Args Yapısı

Komut satırından ayrıştırılan tüm parametreleri saklar:

```
typedef struct {
    Mode    mode;               /* -b veya -a */
    char    input_files[32][256]; /* giriş dosya listesi */
    int     input_count;        /* giriş dosyası sayısı */
    char    output_file[256];    /* çıktı .sau adı */
    char    archive_file[256];   /* açılacak .sau */
    char    dest_dir[256];       /* hedef dizin */
} Args;
```

Önemli Sabitler

Sabit	Değer	Açıklama
MAX_FILES	32	En fazla arşivlenebilecek dosya sayısı
MAX_TOTAL_SIZE	200 MB	Giriş dosyalarının toplam boyut sınırı
HEADER_SIZE_LEN	10	Header boyutunun ASCII alanı uzunluğu (byte)
DEFAULT_OUTPUT	"a.sau"	Çıktı dosyası belirtilmezse kullanılan varsayılan ad
SAU_EXTENSION	".sau"	Geçerli arşiv dosyası uzantısı

.sau Arşiv Dosyası Formatı

Arşiv dosyası iki ana bölümden oluşur: organizasyon (header) bölümü ve arşivlenmiş dosya içerikleri.

Header Bölümü

Header iki kısımdan oluşur:

- İlk 10 byte: Tüm header bölümünün toplam bayt uzunluğunu sağa dayalı ASCII formatında saklar.
- Geri kalan kısım: Her dosya için |dosya_adı,izinler(oktal),boyut| formatında pipe ile ayrılmış kayıtlar içerir.

Dosya İçerikleri Bölümü

Header bölümünün hemen ardından, arşivlenen dosyaların içerikleri herhangi bir ayırıcı kullanılmadan art arda ASCII metin olarak yazılır. Dosyalar, header'daki sıraya göre yerleştirilir; boyut bilgisi açma sırasında sınır tespiti için kullanılır.

Örnek Format Gösterimi

Aşağıda 5 dosya barındıran bir s1.sau arşivinin başlangıç yapısı görülmektedir:

```
73|t1,644,28||t2,644,27||t3,644,27||t4.txt,644,48||t5.dat,644,33|
Bu birinci test dosyasidir.
Bu ikinci test dosyasidir.
...
```

İlk 10 karakter olan "73" header bölümünün toplam 73 byte olduğunu gösterir. Hemen ardından gelen pipe-kayıt bölümünde her dosyanın adı, oktal izinleri ve byte cinsinden boyutu yer alır. 73. byte'tan itibaren dosya içerikleri başlar.

Modül Açıklamaları

parser.c — Komut Satırı Ayırıştırma

parse_args() fonksiyonu argc/argv parametrelerini alarak modu (-b veya -a) belirler ve ilgili parametreleri Args yapısına doldurur.

Pack modunda: -o'dan önce gelen tüm argümanlar giriş dosyası, -o'dan sonraki argüman çıktı dosyası olarak işlenir. Unpack modunda: .sau uzantısı zorunlu olarak doğrulanır; ikinci opsiyonel parametre hedef dizindir.

Doğrulan hata koşulları: yetersiz argüman sayısı, geçersiz mod bayrağı, -a modunda fazla parametre, .sau uzantısı eksikliği, giriş dosyası belirtilmemesi.

pack.c — Arşivleme (-b Modu)

pack_files() ana fonksiyonu üç aşamadan oluşur:

- `validate_and_collect()`: Her giriş dosyasını `stat()` ile kontrol eder, `is_text_file()` ile ASCII doğrulaması yapar, toplam boyutu hesaplar ve Archive yapısını doldurur.
- `build_header()`: Tüm kayıtları `snprintf()` ile birleştirerek header tamponunu oluşturur. İlk 10 byte sabit genişlikte ASCII sayısı içerir.
- `write_archive()`: Çıktı dosyasını açar, header'ı yazar, ardından her dosyanın içeriğini 8 KB'lık bloklar halinde kopyalar.

unpack.c — Arşiv Açma (-a Modu)

`unpack_archive()` ana fonksiyonu üç aşamadan oluşur:

- Arşiv dosyasını `fopen()` ile açar ve hata kontrolü yapar.
- `parse_header()`: İlk 10 byte'tan header boyutunu okur, kayıtları pipe ayrıcısına göre ayrıştırır, `sscanf()` ile ad/izin/boyut alanlarını çıkarır.
- `extract_files()`: Her dosyayı header boyutuna göre konumlandırır, `fread()/fwrite()` ile yazgiya aktarır ve `chmod()` ile orijinal izinleri geri yükler.

utils.c — Yardımcı Fonksiyonlar

- `is_text_file()`: Dosyayı byte byte okuyarak her karakterin yazdırılabilir ASCII (0x20-0x7E) veya izin verilen boşluk karakterleri (`\n`, `\r`, `\t`) olduğunu doğrular.
- `ensure_directory()`: `stat()` ile dizin varlığını kontrol eder; yoksa dizin yolunu `'/'` ile bölerek iç içe `mkdir()` çağrılarını yapar (`mkdir -p` davranışı).
- `print_error()`: Tüm hata mesajlarını `stderr`'e "tarsau: ..." önekiyle yazar.
- `get_file_permissions_str()`: Bir dosyanın izin bitlerini oktal string olarak döndürür.

Derleme

Makefile Yapısı

Proje, GNU make ile derlenebilmek üzere yapılandırılmıştır. Makefile aşağıdaki hedefleri destekler:

Hedef	Açıklama
<code>make / make all</code>	Projeyi tam olarak derler; <code>bin/</code> ve <code>lib/</code> dizinleri yoksa oluşturur
<code>make clean</code>	Tüm <code>.o</code> nesne dosyalarını ve <code>bin/tarsau</code> 'yu siler
<code>make rebuild</code>	<code>clean + all</code> — temiz yeniden derleme

Derleme Komutu ve Çıktı

```
$ make
gcc -Wall -Wextra -I include -c src/main.c      -o lib/main.o
gcc -Wall -Wextra -I include -c src/parser.c    -o lib/parser.o
gcc -Wall -Wextra -I include -c src/pack.c      -o lib/pack.o
gcc -Wall -Wextra -I include -c src/unpack.c    -o lib/unpack.o
gcc -Wall -Wextra -I include -c src/utils.c     -o lib/utils.o
gcc -Wall -Wextra -I include -o bin/tarsau lib/main.o lib/parser.o \
lib/pack.o lib/unpack.o lib/utils.o
```

Test Senaryoları ve Ekran Çıktıları

Programın tüm işlevleri çeşitli test senaryolarıyla doğrulanmıştır. Her testte kullanılan komut ve elde edilen ekran çıktısı aşağıda verilmiştir.

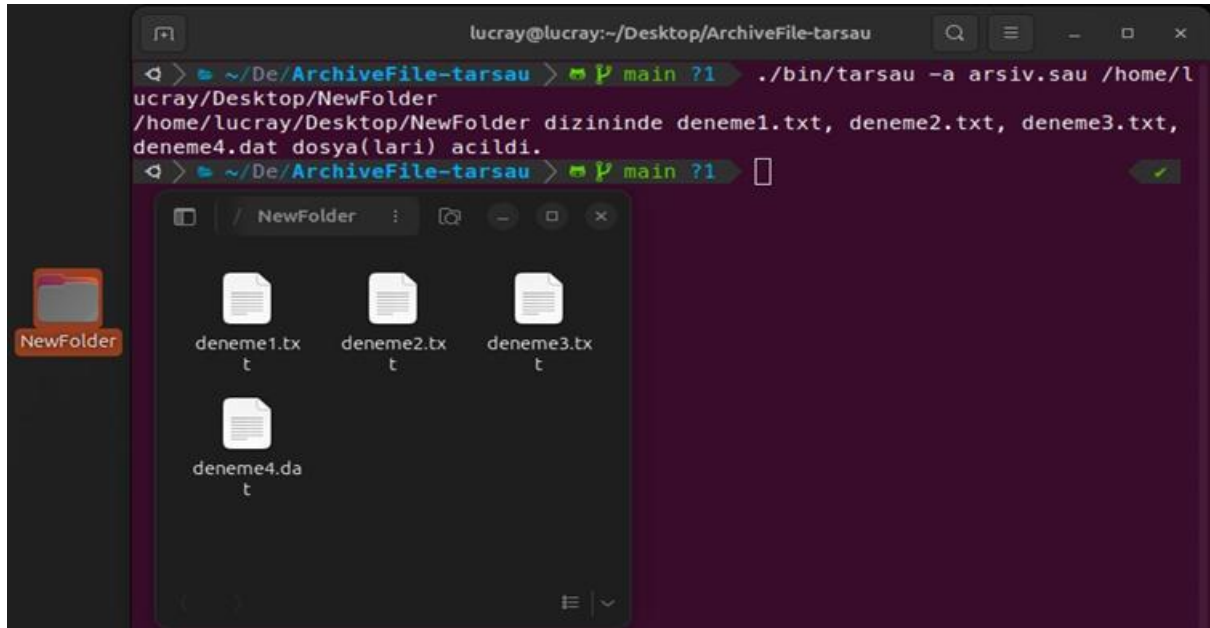
1.Test için /home/lucray/Desktop/deneme/ altında bulunan deneme1.txt deneme2.txt deneme3.txt deneme4.dat dosyaları üzerinde denemeler yapılmıştır. Raporun 3.4 Parametre kısıtlarından dolayı 32 dosyadan fazla ve 200 MB dan büyük dosyalar arşivlenemez. Kısıtlar buna göre oluşturulmuştur.

```
< > ~/Desktop/ArchiveFile-tarsau > main > ./bin/tarsau -b /home/lucray/Desktop/deneme
/deneme1.txt /home/lucray/Desktop/deneme/deneme2.txt /home/lucray/Desktop/deneme/deneme3.txt
/home/lucray/Desktop/deneme/deneme4.dat -o arshiv.sau
Dosyalar birlestirildi.
< > ~/De/ArchiveFile-tarsau > main ?1
```

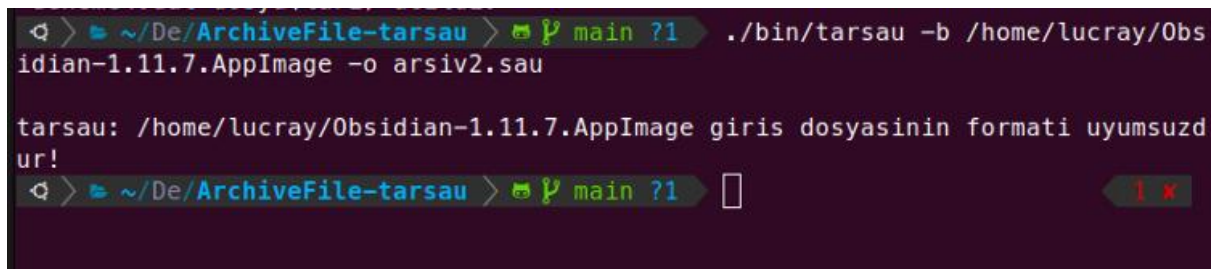
2.Bu kodun işlenmesi sonrasında aşağıdaki çıktı elde edilir. Dosyalar, Dosya adı, izinler, boyut | olarak birbirinden ayrılmıştır.

```
90|deneme1.txt,664,13||deneme2.txt,664,17||deneme3.txt,
664,14||deneme4.dat,664,23|asdfghjkwegr
twretwrevxcvbfgs
bxcvmbvncgfdh
deneme dat dosyasi ile
```

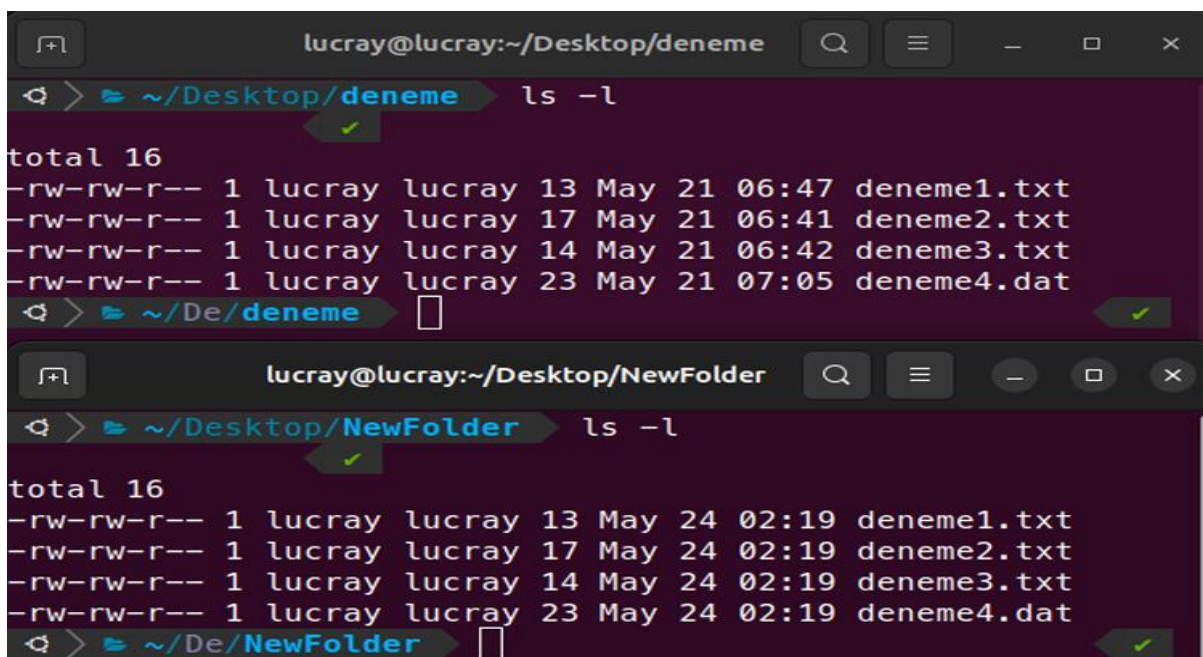
3. Daha sonrasında bu dosyaların /home/lucray/Desktop/NewFolder/ dizinine çıkarılmıştır. NewFolder dizini yoksa oluşturur.



4. Uyumsuz bir dosya girildiğinde aşağıdaki sonuç elde edilir. Program sadece metin dosyaları ile çalışır.



5. Alttağı görselde görüldüğü üzere dosyaların alındığı ve çıkarıldığı dosyalardaki izinleri aynıdır.



Altaki Bölümlerde verilen ekran çıktılarına ek kullanım kısımlarından bahsedilmiştir

Temel Arşivleme Testi (-b)

Beş farklı metin dosyası (t1, t2, t3, t4.txt, t5.dat) tek bir s1.sau arşivinde birleştirildi:

```
$ tarsau -b t1 t2 t3 t4.txt t5.dat -o s1.sau
Dosyalar birlestirildi.
```

Oluşturulan Arşiv Dosyası İçeriği

Arşiv dosyasının ilk bölümü incelendiğinde header formatı açıkça görülmektedir:

```
73|t1,644,28||t2,644,27||t3,644,27||t4.txt,644,48||t5.dat,644,33|
Bu birinci test dosyasidir.
Bu ikinci test dosyasidir.
Bu ucuncu test dosyasidir.
...
```

" 73" değeri header bölümünün toplam 73 byte uzunluğunda olduğunu göstermektedir.

Arşiv Açma Testi (-a)

s1.sau arşivi d1 adlı yeni bir dizine başarıyla açıldı:

```
$ tarsau -a s1.sau d1
d1 dizininde t1, t2, t3, t4.txt, t5.dat dosya(lari) acildi.
```

Açılan dosyaların listesi ve orijinal izinleri korunmuş halde görülmektedir:

```
$ ls -la d1/
total 28
-rw-r--r-- 1 user user 28 May 23 11:28 t1
-rw-r--r-- 1 user user 27 May 23 11:28 t2
-rw-r--r-- 1 user user 27 May 23 11:28 t3
-rw-r--r-- 1 user user 48 May 23 11:28 t4.txt
-rw-r--r-- 1 user user 33 May 23 11:28 t5.dat
```

Mevcut Dizine Açma

Hedef dizin belirtilmeden arşiv mevcut çalışma dizinine açıldı:

```
$ tarsau -a s1.sau
t1, t2, t3, t4.txt, t5.dat dosya(lari) acildi.
```

Varsayılan Çıktı Adı Testi

-o parametresi verilmeden arşivleme yapıldığında varsayılan a.sau dosyası oluşturuldu:

```
$ tarsau -b t1 t2
Dosyalar birlestirildi.
$ ls -la a.sau
-rw-r--r-- 1 user user 87 May 23 11:28 a.sau
```

Hata Durumu — ASCII Olmayan Dosya

İkili (binary) içerikli bir dosya giriş olarak verildiğinde program uyarı mesajı yazarak temiz çıkış yaptı:

```
$ tarsau -b binary_file -o test.sau
tarsau: binary_file giris dosyasinin formati uyumsuzdur!
```

Hata Durumu — Geçersiz Arşiv Uzantısı

.sau yerine farklı bir uzantı kullanıldığında uygun hata mesajı üretildi:

```
$ tarsau -a s1.zip d2
tarsau: Arsiv dosyasi uygunsuz veya bozuk!
```

Geliştirme Süreci ve Kodlar

1. Konsol üzerinden giriş argümanları alınır -a, -b ve diğer parametreler ve uygun fonksiyon çağrıları yapılır. Main sınıfın tek amacı argümanları parse etmek ve uygun çağrıyı yapmaktır.

```
int main(int argc, char *argv[])
{
    Args args;

    if (parse_args(argc, argv, &args) != 0) {
        return EXIT_FAILURE;
    }

    switch (args.mode) {
        case MODE_PACK:
            if (pack_files(&args) != 0)
                return EXIT_FAILURE;
            break;
        case MODE_UNPACK:
            if (unpack_archive(&args) != 0)
                return EXIT_FAILURE;
            break;
        default:
            print_error("Bilinmeyen mod.");
            return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

2. pack.c içinde yazılan diğer yardımcı fonksiyonlarla beraber Uygun dosya içeriğinin hazırlanması işlemi yapılır. Önce Argümanların doğruluğu kontrol edilir ardından dosya adı izni gibi bilgiler ile header kısmı oluşturulur. Dosya içerikler yazılır ve sistem çağrıları ile .sau uzantılı dosya oluşturulur.

```

int pack_files(Args *args)
{
    Archive archive;
    char *header = NULL;
    size_t header_len = 0;
    int ret;

    /* 1. Doğrulama ve bilgi toplama */
    if (validate_and_collect(args, &archive) != 0) {
        return -1;
    }

    /* 2. Header oluştur */
    if (build_header(&archive, &header, &header_len) != 0) {
        return -1;
    }

    /* 3. Arşiv dosyasını yaz */
    ret = write_archive(args->output_file, header, header_len, &archive);
    free(header);

    if (ret != 0) {
        return -1;
    }

    printf("Dosyalar birlestirildi.\n");
    return 0;
}

```

3. unpack.c ile pack işleminde yapılanlar tersine işletilerek dosyalar açılır.

```

int unpack_archive(Args *args)
{
    FILE *fp;
    Archive archive;
    size_t header_size;
    const char *dest_dir;
    int i;

    /* Arşiv dosyasını aç */
    fp = fopen(args->archive_file, "r");
    if (!fp) {
        print_error("Arşiv dosyası uygunsuz veya bozuk!");
        return -1;
    }

    /* Header'ı parse et */
    if (parse_header(fp, &archive, &header_size) != 0) {
        fclose(fp);
        return -1;
    }

    /* Hedef dizin belirle */
    dest_dir = args->dest_dir;

    /* Dizin varsa oluştur */
    if (dest_dir[0] != '\0') {
        if (ensure_directory(dest_dir) != 0) {
            fclose(fp);
            return -1;
        }
    }

    /* Dosyaları çıkar */
    if (extract_files(fp, &archive, dest_dir) != 0) {
        fclose(fp);
    }
}

```

4. parser.c Argümanların parçalanıp modun secilmesi ve moda göre uygun parametereler uygun fonksiyonlara gönderilerek gerekli arşivleme arşivden çıkarma işlemlerini yapar.

```
int parse_args(int argc, char *argv[], Args *args)
{
    /* Yapıyı temizle */
    memset(args, 0, sizeof(Args));
    args->mode = MODE_NONE;
    strcpy(args->output_file, DEFAULT_OUTPUT);

    if (argc < 3) {
        print_error("Kullanım: tarsau -b dosya1 dosya2 ... [-o çıktı.sau]\n"
                    "          tarsau -a arşiv.sau [dizin]");
        return -1;
    }

    /* --- MOD BELİRLE --- */
    if (strcmp(argv[1], "-b") == 0) {
        args->mode = MODE_PACK;
    } else if (strcmp(argv[1], "-a") == 0) {
        args->mode = MODE_UNPACK;
    } else {
        print_error("Gecersiz mod. -b (birleştir) veya -a (aç) kullanın.");
        return -1;
    }

    /* --- PACK MODU ARG PARSE --- */
    if (args->mode == MODE_PACK) {
        int i;
        for (i = 2; i < argc; i++) {
            if (strcmp(argv[i], "-o") == 0) {
                /* -o'dan sonra çıktı dosya adı gelmeli */
                if (i + 1 >= argc) {
                    print_error("-o parametresinden sonra dosya adı belirtilmelidir.");
                    return -1;
                }
                strcpy(args->output_file, argv[i + 1]);
                args->output_file[sizeof(args->output_file) - 1] = '\0';
            }
        }
    }
}
```

Proje, GitHub üzerinde sürüm kontrollü olarak geliştirilmiştir. Geliştirme süreci 3 commit ile 2 geliştirici arasında paylaşılmıştır.

Aşama 1 — Temel Arşivleme Çekirdeği

İlk aşamada projenin en kritik kısmı olan arşivleme (pack.c) ve arşiv açma (unpack.c) modülleri geliştirildi. Aynı zamanda Makefile oluşturularak derleme altyapısı kuruldu.

Bu aşamada gerçekleştirilen temel işlevler:

- validate_and_collect(): Giriş dosyalarını stat() ve is_text_file() ile doğrulama, boyut limiti kontrolü
- build_header(): |dosya_adı,izinler,boyut| formatında header tamponu oluşturma
- write_archive(): Header + dosya içeriklerini sırayla çıktı dosyasına yazma
- parse_header(): .sau dosyasının ilk 10 byte'ından header boyutunu okuma, pipe-ayrılmış kayıtları ayrıştırma
- extract_files(): Kayıtlardaki boyut bilgisine göre dosya içeriklerini okuma ve chmod() ile izin yükleme

Bu aşamadaki kritik tasarım kararı: dosya içeriklerini herhangi bir ayırıcı kullanmadan art arda yazmak ve sınır tespitini yalnızca header'daki boyut alanıyla yapmak. Bu yaklaşım formatı sade tutar ve ayrıştırmayı hızlandırır.

Aşama 2 — Dokümantasyon

İkinci aşamada README.md dosyası oluşturularak projenin derleme, kullanım ve klasör yapısı belgelenmiştir. Bu belge, projeyi ilk kez kullananların hızlıca başlayabilmesi için gerekli tüm komutları içermektedir.

Aşama 3 — Komut Satırı Arayüzü ve Yardımcı Modüller

Son aşamada projenin giriş noktası (main.c), komut satırı ayrıştırıcısı (parser.c), yardımcı fonksiyonlar (utils.c) ve tüm modüllerin paylaştığı ortak başlık dosyası (tarsau.h) geliştirildi.

Bu aşamada gerçekleştirilen işlevler:

- `parse_args()`: -b ve -a modlarını ve tüm parametrelerini doğrulayan komut satırı ayrıştırıcısı
- `is_text_file()`: Byte-byte ASCII doğrulaması; 0x7F üzeri veya izin verilmeyen kontrol karakterleri içeren dosyaları reddeder
- `ensure_directory()`: `mkdir -p` benzeri iç içe dizin oluşturma
- `print_error()`: Tüm modüllerin kullandığı merkezi hata raporlama
- `tarsau.h`: `FileEntry`, `Archive`, `Args` yapıları ve `MAX_FILES`, `HEADER_SIZE_LEN` gibi sabitlerin tek noktada tanımlanması

Bu aşamada alınan önemli bir karar, tüm veri yapılarının `tarsau.h`'de toplanmasıdır. Böylece modüller arasındaki bağımlılıklar net biçimde yönetilmiş ve header döngüsel bağımlılığı önlenmiştir.

Sonuç

Bu projede Linux ortamında C dilinde çalışan, tar/zip benzeri bir arşivleme aracı başarıyla geliştirilmiştir. Program, ASCII metin dosyalarını tek bir .sau arşivinde birleştirmekte ve arşivi orijinal dosya izinleriyle birlikte geri açabilmektedir.

Proje kapsamında komut satırı argüman ayrıştırma, özel dosya formatı tasarımı, POSIX sistem çağrıları, modüler kod organizasyonu ve Makefile kullanımı gibi sistem programlama konularında pratik deneyim edinilmiştir. Tüm hata durumları ele alınmış, program herhangi bir çökme olmaksızın anlamlı hata mesajları üretmektedir.